

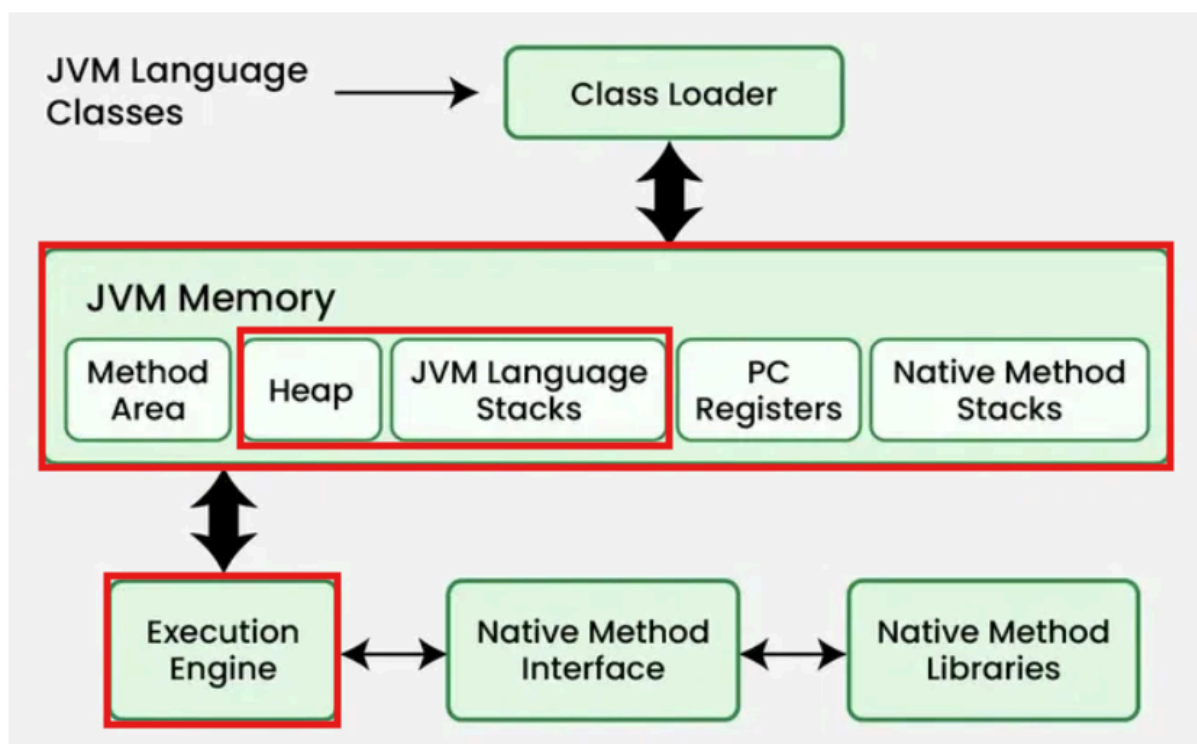
Agenda:

1. Java Memory Model và Garbage Collection

- Stack vs Heap
- Cơ chế cấp phát bộ nhớ cho object
- Garbage Collector hoạt động như thế nào
- Các loại GC trong JVM (G1, ZGC...)

Demo: Tạo chương trình sinh nhiều object và quan sát GC.

1. Bộ nhớ Stack & Heap
2. Cấp phát bộ nhớ cho object
3. GC hoạt động như thế nào
4. Các loại GC
5. Kịch bản demo



JVM quản lý 5 vùng nhớ, nhưng có 3 vùng rất khó can thiệp. Vùng Heap và JVM Language Stacks chịu tác động trực tiếp từ code của lập trình viên. Garbage Collector (cùng với JIT Compiler) là một module cốt lõi nằm chìm bên trong khối Execution Engine.

I. Stack vs Heap

[Java Memory Management - GeeksforGeeks](https://www.geeksforgeeks.org/java-memory-management/)

How are Java Objects Stored in Memory? - GeeksforGeeks

1. JVM Stack Memory

Lưu trữ: Method frames, local primitive variables và references

Đặc tính: Mỗi thread có một Stack **riêng biệt** => **Thread-safe**

Cơ chế:

- Hoạt động theo **LIFO**. Các frame được cấp phát khi lời gọi hàm bắt đầu, và sẽ được giải phóng khi kết thúc hàm.
- Quản lý cấp phát bằng hai con trỏ stack: stack pointer SP và frame pointer FP. Khi một hàm được gọi, FP sẽ dịch chuyển đến đỉnh của stack, cho biết địa chỉ bắt đầu của frame mới; SP dịch chuyển thêm 1 đoạn cần cấp phát.

Giải phóng: Tự động khi method kết thúc (**pop stack frame**).

Hiệu năng: Cực kỳ **nhANH** (nhờ dịch chuyển SP và tối ưu CPU Cache).

Giới hạn: Kích thước **nhỏ**, thường là **cố định** (trên Linux/Windows 64-bit thường mặc định là 1MB cho mỗi thread) => Gây lỗi **StackOverflowError** nếu đệ quy quá sâu.

Có thể quy định kích thước của stack dành cho mỗi thread khi khởi chạy bằng cờ -Xss. VD: **java -Xss256k -jar app.jar**

2. Heap Memory

Lưu trữ: Objects và Arrays sinh ra từ từ khóa **new**.

Đặc tính: Dùng chung giữa tất cả các threads => cần **đồng bộ hóa** để tránh xung đột.

Cơ chế: Cấp phát bằng cách tìm vùng nhớ trống tiếp theo hoặc đã bị sử dụng. Truy xuất theo **Random Access**.

Giải phóng: **KHÔNG** tự động ngay lập tức, mà phụ thuộc hoàn toàn vào tiến trình **Garbage Collector (GC)**.

Hiệu năng: **Chậm hơn** (do tốn chi phí tìm kiếm bộ nhớ, chạy GC, và dễ bị CPU Cache Miss).

Giới hạn: Kích thước **lớn**, dung lượng **động** => Đầy sẽ gây lỗi **OutOfMemoryError**.

Quy định kích thước của heap bằng các cờ **-Xms** (Initial Heap Size) và **-Xmx** (Max Heap Size). **VD:** **java -Xms512m -Xmx2g -jar app.jar**
(Thiết lập RAM khởi điểm 512MB, tối đa 2GB).

Nếu không khai báo trực tiếp thì giá trị mặc định phụ thuộc vào:

- JVM implementation (HotSpot, OpenJ9...)
- Phiên bản Java
- Container (Docker, Kubernetes)

Heap memory được chia thành các vùng:

- **Young Generation:** vùng heap nơi các đối tượng mới thường được cấp phát. Gồm 3 vùng:
 - Eden Space: nơi object mới được tạo
 - Survivor 0 (S0)
 - Survivor 1 (S1)
 - Cấp phát theo kiểu bump-the-pointer (dịch chuyển con trỏ Young Generation, gần giống stack)
- **Old Generation:** vùng heap nơi các đối tượng tồn tại lâu dài được lưu trữ sau khi đã vượt qua nhiều vòng garbage collection.
 - Mỗi object có "age" (GC age)
 - Sau mỗi lần sống sót qua GC => age tăng
 - Khi đạt ngưỡng (MaxTenuringThreshold) => chuyển sang Old Gen
 - Cần cơ chế phức tạp (free list, compaction)
- **Permanent Generation:** Lưu trữ metadata về class và method trong Java 7 trở về trước. => Sau này bị thay thế bằng *metaspace* (**vùng này nằm ngoài heap**)

3. Lý do stack nhanh hơn heap

Stack (Nhờ LIFO): Việc cấp phát hay giải phóng bộ nhớ cực kỳ đơn giản. CPU chỉ cần thực hiện một phép toán duy nhất: **cộng hoặc trừ một thanh ghi con trỏ (Stack Pointer)**. Khi một hàm được gọi, con trỏ tịnh tiến lên để tạo không gian. Khi hàm kết thúc, con trỏ lùi lại, vùng nhớ tự động bị ghi đè vào lần sau.

Heap (Random Access & Shared): Việc cấp phát một object trên Heap tốn kém hơn nhiều. JVM phải thực hiện các bước:

1. Tìm kiếm một khối nhớ (block) đủ lớn và còn trống trong vùng Heap (dù có Thread-Local Allocation Buffer hỗ trợ thì vẫn phức tạp hơn Stack).
2. Tính toán kích thước Object Header, căn chỉnh byte (padding).

3. Xử lý vấn đề Synchronization vì Heap được chia sẻ giữa nhiều threads.
4. Giải phóng thì lại phải chờ Garbage Collector (GC).

II. Cơ chế cấp phát bộ nhớ cho object

[Java Objects Inside Out](#)

[Thread-Local Allocation Buffers in JVM](#)

[Java HotSpot Virtual Machine Performance Enhancements](#)

[Oracle - Managing Memory and Garbage Collection](#)

1. Tính toán kích thước

JVM tính toán tổng bộ nhớ cần thiết bao gồm:

- **Object Header:** Chứa Mark Word (lưu thông tin lock, hashCode, GC age) và Class Pointer (trỏ đến metadata ở Method Area).
- **Instance Data:** Kích thước thực tế của các field.
- **Padding:** Căn chỉnh bộ nhớ (thường là bội số của 8 bytes) để CPU đọc nhanh hơn.

2. Chiến lược cấp phát

- **Ưu tiên 1 - TLAB (Thread-Local Allocation Buffer):** JVM chia Heap thành nhiều TLAB nhỏ, mỗi thread được cấp 1 TLAB riêng (nằm trong Eden space). Object được cấp phát trực tiếp vào TLAB của thread đang thực thi. Quá trình này không cần synchronization, chỉ tịnh tiến con trỏ nếu TLAB còn chỗ.
- **Ưu tiên 2 - Xử lý nếu TLAB không đủ chỗ:** Nếu TLAB đầy hoặc object lớn hơn TLAB, JVM thực hiện 1 trong 2 lựa chọn:
 - Cấp TLAB mới cho thread
 - Allocate trên Eden space và được quản lý đồng bộ giữa các thread, dùng cơ chế CAS (Compare-And-Swap)
- Lựa chọn option nào để thực thi phụ thuộc vào size của object. Nếu object có size lớn hơn N% của TLAB (`-XX:TLABWasteTargetPercent=N`, default `N = 1`), JVM chọn allocate vào Eden, nếu không thì cấp TLAB mới.
- **Ưu tiên 3 - Old Generation:** Nếu đối tượng quá lớn (ví dụ mảng byte vài chục MB), nó sẽ bỏ qua vùng Young và nhảy thẳng vào Old Generation để tránh chi phí copy qua lại vô ích giữa các phân vùng.

=> Young Generation được tối ưu cho throughput, Old Generation được tối ưu cho memory efficiency

3. Escape Analysis

Là một kỹ thuật tối ưu của JVM để xác định Object có "thoát ra ngoài" phạm vi method/thread hay không

Nếu không escape:

- Có thể cấp phát trên Stack thay vì Heap
- Có thể loại bỏ hoàn toàn object (Scalar Replacement)
- Có thể loại bỏ synchronization

=> Giúp giảm GC pressure và tăng hiệu năng đáng kể

VD:

```
int foo(int z) {  
    MyObject obj = new MyObject();  
    obj.x = z + 1;  
    return obj.x;  
}
```

=> obj chỉ dùng trong method, không return, không gán ra ngoài qua object khác

Cùng đoạn code trên nếu được thực hiện trong chương trình đủ nhiều thì JIT Compiler chạy Escape Analysis xong áp dụng Scalar Replacement, "xé" object *obj* thành primitive-type variable và cấp phát trực tiếp trên stack. Sau khi tối ưu hóa thì compiler sẽ ngầm hiểu thành:

```
int foo(int z) {  
    int x = z + 1; // Giả sử thuộc tính x của obj ban đầu  
    mang kiểu int  
    return x;  
}
```

III. Garbage Collector hoạt động như thế nào

[Garbage Collection in Java - GeeksforGeeks](#)

[Mastering Garbage Collection in Java: From Beginner to Expert | by Arvind Kumar](#)

GC hoạt động dựa trên cốt lõi là **Weak Generational Hypothesis** (Giả thuyết thế hệ): Hầu hết các object sinh ra đều trở thành rác rất nhanh.

1. Thuật toán cốt lõi (Reachability Analysis & Mark-Sweep-Compact)

- **Root Search:** GC bắt đầu rà soát từ các "rễ" (Local variables trên Stack, Active Threads, Static fields, JNI references).
- **Reachability Analysis:** Duyệt theo đồ thị tham chiếu từ GC Roots, nếu không có đường đi tới object từ root, thì object đó được coi là "rác".
- **Mark:** Đánh dấu tất cả reachable object.
- **Sweep:** Xóa rác
- **Compact (Optional):** dồn các object còn sống lại gần nhau để chống phân mảnh bộ nhớ (Memory Fragmentation).

2. Luồng dịch chuyển đối tượng (Generational GC)

Thiết kế dựa trên triết lý "*Most objects die young*"

- **Minor GC:** Khi Eden đầy, Minor GC kích hoạt. Các object sống sót được copy sang vùng Survivor (S0 hoặc S1). Tuổi của object tăng lên 1. Gây ra hiện tượng **Stop-The-World** (toàn bộ thread của ứng dụng bị đóng băng để an toàn thu gom rác) trong thời gian **ngắn** (vài ms).
- **Promotion:** Object sống qua nhiều đợt Minor GC (đạt ngưỡng **MaxTenuringThreshold**, mặc định HotSpot là 15) sẽ được đẩy lên vùng Old Generation.
- **Major GC / Full GC:** Quá trình dọn rác ở đây tốn rất nhiều thời gian và gây ra hiện tượng **STW dài**. Xảy ra trong các trường hợp:
 - Old Generation đầy, Metadata (Metaspace) đầy
 - System.gc() được gọi
 - Allocation failure.

IV. Các loại GC (hoặc thuật toán ?) trong JVM

1. Serial GC

Tổng quan về Serial GC:

- Nó sử dụng một luồng (thread) cho việc thu thập rác và dừng toàn bộ ứng dụng trong quá trình thu gom rác, gọi là stop-the-world (STW)
- Khi thực hiện STW, toàn bộ ứng dụng sẽ bị tạm dừng, bao gồm cả luồng chính (main thread), cho đến khi quá trình thu gom rác hoàn tất. Kết quả là tất cả các luồng (thread) đều bị tạm dừng trong khoảng thời gian ngắn.

- Serial GC được sử dụng mặc định trong các ứng dụng Java nhỏ và đơn giản, không yêu cầu xử lý dữ liệu lớn và đòi hỏi tài nguyên bộ nhớ thấp.

Quá trình thu gom rác được thực hiện theo hai giai đoạn:

- Giai đoạn Mark: đánh dấu (mark) các đối tượng được sử dụng bằng cách đệ quy thông qua tất cả các đối tượng bắt đầu từ đối tượng root (các biến static, biến local, và đối tượng bắt đầu từ stack).
- Giai đoạn Sweep: quét toàn bộ heap và giải phóng bộ nhớ của các đối tượng không được đánh dấu (unmarked).

Hạn chế:

- Tổng thời gian thực hiện quá trình thu gom rác của thuật toán Serial GC sẽ tăng theo tỷ lệ với kích thước bộ nhớ heap, và đặc biệt là trong giai đoạn Sweep.
- Khi thực hiện quá trình Sweep, JVM phải tìm kiếm toàn bộ heap để giải phóng bộ nhớ của các đối tượng không được đánh dấu, điều này có thể làm tăng độ phức tạp và thời gian thực hiện của quá trình thu gom rác.

2. Concurrent Mark Sweep (CMS) GC

Tổng quan:

- Concurrent Mark Sweep (CMS) GC là một thuật toán thu gom rác được thiết kế để giảm thiểu độ trễ (latency) trong quá trình thu gom rác bằng cách sử dụng nhiều luồng để thực hiện việc thu gom rác.
- CMS GC được sử dụng phổ biến trong các ứng dụng yêu cầu thời gian đáp ứng nhanh và có thời gian dừng (stop-the-world time) thấp.

CMS GC được thực hiện trong các giai đoạn sau:

- Initial Mark: JVM tạm dừng tất cả các luồng thực thi và xác định các đối tượng trực tiếp được tham chiếu bởi các biến tĩnh, biến stack và các thanh ghi CPU. Các đối tượng này được đánh dấu là "live".
- Concurrent Mark: JVM bắt đầu chạy các luồng thu gom rác để xác định các đối tượng "live" khác bằng cách truy xuất các đối tượng đã được đánh dấu trong Initial Mark. Quá trình này được thực hiện song song với luồng chính của ứng dụng.
- Remark: JVM tạm dừng tất cả các luồng thực thi và xác định các đối tượng "live" mới được tham chiếu trong quá trình Concurrent Mark.
- Concurrent Sweep: JVM bắt đầu chạy các luồng thu gom rác để giải phóng các đối tượng không còn được sử dụng. Quá trình này được thực hiện song song với luồng chính của ứng dụng.

Hạn chế:

- Việc sử dụng nhiều luồng trong quá trình thu gom rác có thể làm tăng độ phức tạp của thuật toán và tốn nhiều tài nguyên hệ thống.
- CMS GC cũng có thể gây ra tình trạng Fragmentation (phân mảnh) bộ nhớ, dẫn đến sự lãng phí tài nguyên và giảm hiệu suất của ứng dụng.

3. Parallel GC**Tổng quan:**

- Parallel GC là một thuật toán GC sử dụng multiple threads để thu thập rác trên bộ nhớ heap. Nó được thiết kế để tận dụng các bộ vi xử lý có nhiều lõi, giúp tăng tốc độ thu gom rác bằng cách sử dụng nhiều thread đồng thời.
- Parallel GC có thể được cấu hình để sử dụng một số lượng thread cố định hoặc tự động điều chỉnh số lượng thread dựa trên số lượng CPU hiện có. Điều này giúp tối ưu hóa hiệu suất thu gom rác và giảm thiểu độ trễ của ứng dụng.

Parallel GC được thực hiện trong các giai đoạn sau

- Trong giai đoạn đầu tiên, các đối tượng được đánh dấu để xác định những đối tượng nào sẽ bị thu gom.
- Trong giai đoạn thứ hai, các đối tượng được thu gom và bộ nhớ của chúng được giải phóng.

Hạn chế:

- Do sử dụng nhiều thread để thực hiện thu gom rác, Parallel GC có thể gây ra tình trạng sử dụng CPU cao và gây ảnh hưởng đến hiệu suất của các tiến trình khác trên hệ thống.

4. Garbage First (G1) GC**Tổng quan:**

- G1 GC được thiết kế để giải quyết các vấn đề về hiệu suất và khả năng mở rộng của CMS GC, đồng thời giảm thiểu độ trễ (latency) trong quá trình thu gom rác.

Đặc điểm:

- Thuật toán G1 GC được chia thành các vùng (regions) bằng cách chia bộ nhớ heap thành các khu vực bằng nhau.

- Khi quá trình thu gom rác bắt đầu, G1 GC sẽ tập trung vào các vùng có nhiều đối tượng rác, đảm bảo rằng các vùng này sẽ được thu gom trước. Điều này giúp giảm thiểu độ trễ và tối ưu hóa việc sử dụng bộ nhớ.
- G1 GC sử dụng các vòng lặp thu gom rác ngắn hơn và phân tán các hoạt động thu gom rác trong nhiều chu kỳ thu gom rác. Các chu kỳ này được điều khiển bởi JVM và được thiết kế để đảm bảo rằng các vùng được thu gom rác trước khi cạn kiệt bộ nhớ.
- G1 GC có thể giảm thiểu độ trễ và tăng hiệu suất bằng cách cho phép xử lý đa luồng trong quá trình thu gom rác. Nó cũng có khả năng xử lý các tệp heap lớn hơn mà không ảnh hưởng đến hiệu suất.

Hạn chế:

- G1 GC có thể tốn nhiều thời gian để tối ưu hóa và phù hợp với các ứng dụng có yêu cầu bộ nhớ heap lớn. Nó cũng có thể ảnh hưởng đến hiệu suất nếu các vùng được chia quá nhỏ hoặc nếu số lượng đối tượng rác quá ít.

5. Z Garbage Collector (ZGC)

Tổng quan:

- ZGC được thiết kế để hỗ trợ các ứng dụng Java đòi hỏi tính khả dụng cao và thời gian phản hồi thấp. Điều này đặc biệt quan trọng trong các ứng dụng có thời gian phản hồi thấp, chẳng hạn như các ứng dụng trò chơi hoặc các ứng dụng xử lý dữ liệu theo thời gian thực.
- Thuật toán ZGC được thiết kế để hoạt động với các bộ nhớ rất lớn, đến hàng trăm gigabyte hoặc thậm chí là terabyte. Nó sử dụng một số kỹ thuật mới như tối ưu hóa bộ nhớ và điều khiển tỉ lệ bộ nhớ, và đồng thời vẫn duy trì khả năng tùy chỉnh cao.

Đặc điểm, tính năng:

- Tự động điều chỉnh kích thước bộ nhớ để đảm bảo tính khả dụng cao.
- Giảm đáng kể thời gian phản hồi trong quá trình thu gom rác.
- Duy trì khả năng tùy chỉnh cao, cho phép người dùng điều chỉnh các tham số để phù hợp với nhu cầu của ứng dụng.
- Hỗ trợ các hệ thống có nhiều nhân xử lý (multithreading), giúp tăng tốc độ xử lý.

Hạn chế:

- ZGC cũng có một số hạn chế, chẳng hạn như tốc độ thu gom rác không bằng G1 GC và phải sử dụng một số lượng bộ nhớ tạm để lưu trữ các đối tượng được gán nhãn.

6. Shenandoah GC

Tổng quan:

- Shenandoah GC là một thuật toán GC thuộc loại concurrent, có khả năng giảm thiểu độ trễ và tối ưu hóa hiệu suất của ứng dụng.
- Shenandoah GC được thiết kế để hoạt động với bất kỳ kích thước heap nào và có khả năng đáp ứng yêu cầu của các ứng dụng với tải lớn và độ trễ thấp. Nó hoạt động bằng cách chia heap thành các vùng con và thực hiện thu gom rác trên từng vùng con một.

Đặc điểm

- Thuật toán Shenandoah GC sử dụng một phương pháp đánh dấu trường hợp đặc biệt để xác định các đối tượng có thể bị thu gom mà không cần quét toàn bộ heap. Nó sử dụng một bảng tham chiếu để giữ các tham chiếu đến các đối tượng được đánh dấu và sử dụng bảng này để quét các vùng con của heap.
- Shenandoah GC có khả năng hoạt động đồng thời với các luồng thực thi của ứng dụng, vì vậy nó không gây ra độ trễ đáng kể trong quá trình thu gom rác. Nó cũng có khả năng thích ứng với các trường hợp tải lớn và độ trễ nhỏ mà không cần thiết lập các thông số GC trước đó.

Hạn chế:

- Shenandoah GC có thể tăng độ trễ trong quá trình khởi động của JVM và có thể yêu cầu bộ nhớ phụ trợ để lưu trữ bảng tham chiếu. Nó cũng có thể gây ra độ trễ nhất định trong quá trình xử lý các đối tượng được đánh dấu và trong quá trình compacting heap.